

/A1 Group 8

Giacomo Cappelletto, Ava Johnson,

Birinder Bachhal, Ceren Atalar

The Deskinator

## **Final Design Report**

### **Executive summary**

This project focuses on developing a fully autonomous desktop-cleaning robot (“Deskinator”) designed to maintain the EK131–210 classroom workspaces between student uses. The goal is to provide a reliable, debris-removing solution that reduces the need for manual cleaning, ensures a sanitary environment, and increases classroom efficiency. The device is engineered to operate safely on a 4 ft x 4 ft desk surface, navigate without falling off, and remove debris, represented by grains of rice in testing, using a custom-built vacuum system.

To achieve this goal, the Deskinator integrates two major subsystems: a motion platform capable of detecting desk boundaries using onboard proximity sensors, and a vacuum mechanism designed specifically for this project. These are orchestrated by software running on a Raspberry Pi 4B, which performs real-time localization via an Extended Kalman Filter, boundary discovery through a bump-and-turn state machine, and boustrophedon coverage planning, all visualizable remotely over SSH/VNC. The robot is powered entirely by an onboard battery and is triggered using a touchless activation method, such as a hand wave. Upon completing its cleaning cycle, it provides a clear end-of-task signal through an led.

Testing evaluated the robot’s ability to safely move about the desk surface, successfully detect edges, collect debris from the safe operation area into a removable sealed container, operate wirelessly for more than five minutes, and complete its cleaning task within the required two minutes. Results demonstrated that the robot met the performance criteria: it navigated without falling, reliably responded to the touchless gesture start signal, collected an average of 99.40% of rice grains with the custom vacuum into the attached container, and indicated initiation and completion through the LED.

### **Introduction**

#### **Problem Statement**

The goal is to design a device that can clean and maintain shared EK131-210 desks in between uses, to ensure that students have a debris-free and sanitary workspace, without requiring a staff to manually wipe down each desk during classes.

## Key Objectives and Metrics

| Objective              | Metric (constraint)                                                                                    |
|------------------------|--------------------------------------------------------------------------------------------------------|
| Reliable               | Successfully complete 99% of cleaning cycles without malfunction or human intervention.                |
| Accuracy               | Remove 95% of rice from the 4x4 desk area.                                                             |
| Efficiency             | Clean the full desk area in under 2 minutes.                                                           |
| Automated              | Must operate entirely on its own after receiving the start signal.                                     |
| Navigation             | Must navigate the desk edges without falling off.                                                      |
| Durability             | Must withstand day to day use.                                                                         |
| Easy to use            | A new user must be able to start cleaning without much instruction.                                    |
| Touchless              | Must start and stop without physical contact.                                                          |
| Storage capacity       | The bin must hold rice from three full cleaning cycles before requiring emptying.                      |
| Precision              | The robot's motion maintains consistent tracking accuracy, with cleaning paths within a 1.5 cm margin. |
| Affordability          | The total material cost must remain under \$200.                                                       |
| Portable               | The robot must weigh less than 750g for easy transportation.                                           |
| Easy to clean          | Dustbin and brushes must be removable and washable.                                                    |
| Compact                | The robot must not exceed 12in x 12in to fit on standard desks.                                        |
| Completion recognition | Must signal task completion via light or sound.                                                        |

## Innovative Elements

### Software

#### Extended Kalman Filter EKF for Localization

The EKF in slam/ekf.py is the foundation of the robot's self-awareness on the table. It maintains a probabilistic estimate of the robot's pose ( $x$ ,  $y$ ,  $\theta$ ) by fusing two complementary sensor streams,

which are the wheel odometry from the stepper motors and yaw rate from the MPU-6050 gyroscope. The prediction step uses differential drive kinematics to propagate position based on wheel displacements, while the gyro update corrects angular drift that accumulates from wheel slip. The filter also implements tactile localization via `update_line_constraint()`. This happens when the robot detects a table edge during the coverage phase through the APDS9960 front facing sensors, it therefore knows it must be on the boundary of the table and snaps its position to that known wall. This closed-loop correction keeps cumulative drift bounded even during long cleaning sessions by enforcing known constraints (eg. table edges are at  $90^\circ$ ) to the pose graph estimation model.

### Wall Follower with Bump-and-Turn Logic

The `WallFollower` in `control/wall_follower.py` implements a boundary discovery algorithm using a simple but effective bump-and-turn state machine. The robot drives forward until the APDS-9960 proximity sensors detect an edge (raw value drops below threshold in `config.py`, meaning no surface reflection), then executes a choreographed sequence: back up a fixed distance, rotate a few degrees opposite to the detected side, and continue. By counting corners and tracking distance traveled, the algorithm can detect when it has completed a full lap around the table (4 corners + returning near the start pose). This approach is very tolerant of sensor noise, because rather than trying to maintain precise wall-following distance, it embraces contact with the boundary as the primary source of information. The corner poses collected during this phase are fed to `RectangleFit` to estimate the table's dimensions using an implementation of RANSAC, and a loop closure is added to the pose graph when the lap completes, globally correcting accumulated drift.

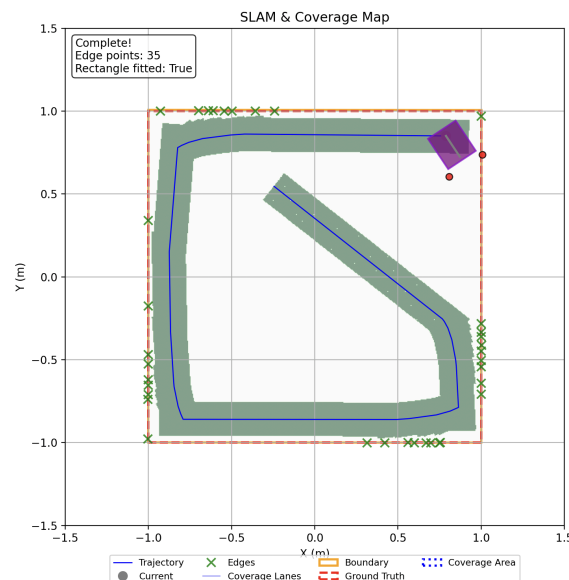


Figure 1: Example of Wall Follower used to compute boundary based on found edges

### Boustrophedon Coverage Planning with Real-Time Visualization

Once the boundary is known, the CoveragePlanner in `planning/coverage.py` generates a boustrophedon (lawn-mower) pattern of parallel lanes that guarantees complete coverage. It computes lane spacing based on the vacuum width minus a configurable overlap (in `config.py`), ensuring no gaps between passes and a fully swept table. The lanes alternate direction ("back-and-forth") to minimize unnecessary turns. Meanwhile, the SweptMap tracks which cells have been visited, and the Visualizer in `utils/viz.py` renders both the SLAM map (showing trajectory, detected edges, loop closures, and tactile corrections) and a green heatmap of coverage in real-time using matplotlib. This immediate feedback lets users verify the robot is working correctly and see exactly which areas have been cleaned.

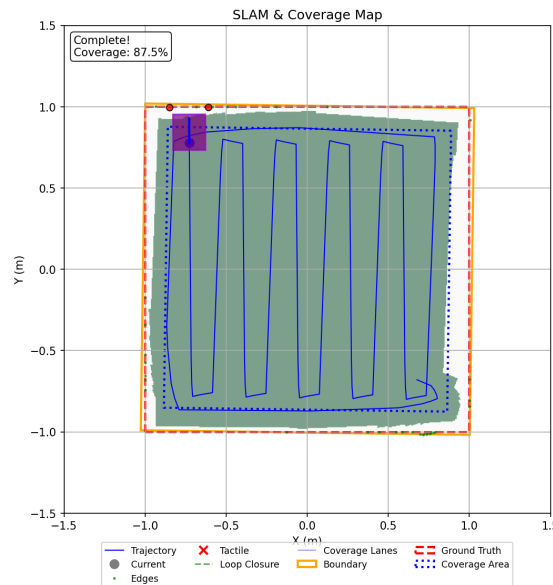


Figure 2: Example of computed and followed Boustrophedon path

## **Hardware**

### Raspberry Pi 4B

Instead of an Arduino-based microcontroller, the Deskinator uses a Raspberry Pi 4B for three reasons. First, its Linux environment enables SSH/VNC connectivity, streaming the real-time pose graph and coverage heatmap to a remote computer without onboard display hardware. Second, the Pi supports multiple software I2C buses, which is critical for the three APDS-9960 sensors that share the same fixed address (0x39), avoiding external multiplexers. Third, networked Python with asyncio allows a single workstation to monitor and control multiple

robots simultaneously, enabling a future implementation of fleet-scale operation across several desks.

### Modular Battery Pack

Because the robot is designed for repeated, extended cleaning cycles on frequently used tables, we implemented a fully external, hot-swappable battery system. The two Li-ion battery packs mounted on the top of the housing provide approximately 3 hours of continuous operation (see Appendix A.8). This modular design allows one pack to power the robot while the other is charging, enabling uninterrupted use and minimizing downtime during long cleaning sessions.

### IMU - MPU6050

To ensure accurate dead reckoning and therefore enhance precision in the loop closures of the SLAM pose graph estimation algorithm, the use of a 6-axis gyro and accelerometer was employed. The fusion of the MPU6050's data (bearing, angular acceleration, linear acceleration) and the motor's calculated odometry ensures that errors linked to drift and slip are reduced to a minimum, and loop closures yield tighter data in the algorithm part.

## **Final Product**

The final product is a fully autonomous desktop-cleaning robot that combines advanced software, precision hardware, and coordinated sensing actuation to deliver a cleaning system. Its software integrates a bump-and-turn wall-following algorithm to map out boundaries, a boustrophedon coverage planner that generates parallel lanes without gaps, and an Extended Kalman Filter SLAM framework that fuses wheel odometry from two NEMA17 motors with data from the MPU6050 and edge constraints from the APDS9960 sensors. Together these ensure drift bounded localization throughout both mapping and cleaning phases. The hardware system includes a dual-battery pack for extended operation, high-torque stepper motors for consistent differential drive motion, and a custom vacuum chute built using the continuity principle (3) to amplify suction velocity at the intake. The APDS9960 sensors enable both reliable edge detection and touchless activation, while the Raspberry Pi 4B handles multi-bus I2C communication, real-time visualization, and remote SSH access.

The code supporting these subsystems is organized into modular components that define the robot's behavior. Low-level modules control the motors and vacuum, while sensor drivers manage IMU data, APDS9960 proximity readings, and gesture detection across separate software-I2C buses. SLAM and mapping modules implement the EKF prediction-update cycles, boundary fitting, and edge-based corrections, while the planning layer handles wall following, lane generation, and sweep-map coverage tracking. Testing and simulation tools support repeated trials, generate SLAM and coverage visualizations, and verify system performance. Altogether, this integrated software manages actuator behavior, sensor processing, boundary estimation,

coverage execution and simulation-based testing, resulting in a platform capable of safely navigating a desktop and removing debris.

$$V_1 A_1 = V_2 A_2 \quad (3)$$

Where: V = Velocity, A = Area

## Results

### Coverage Testing

#### Testing Design

The robot's cleaning performance was evaluated through a comprehensive testing framework implemented in `robot_testing.py`, which executed 50 independent trials of the complete cleaning cycle of a 2x2m desk, of which 2 quit unexpectedly, leaving 48 full completed trials. This converts to a 96% cleaning trial efficacy, which is close to the Key Objective of 99% eff. Each trial consisted of two phases: boundary discovery, where the robot employed wall-following behavior to map the table perimeter and fit a rectangular boundary model, and coverage execution, where the robot followed a systematic boustrophedon path pattern to clean the identified region. During each trial, eight key performance metrics were collected, including edge points detected, rectangle fitting error (measured in parts per million), coverage percentage for both the full and inset rectangles (safety for edges), boundary discovery time, coverage time, total cleaning time, and total distance traveled.

All trial data was exported to Excel spreadsheets containing both raw measurements and summary statistics (mean, standard deviation, minimum, and maximum values). The results were subsequently analyzed using `visualize_results.py`, which generated Gaussian distribution plots for each performance metric by fitting normal distributions to the empirical data. This visualization script produced individual histograms overlaid with fitted Gaussian curves showing the mean and standard deviation for each metric, along with a comparative time analysis plot displaying the distributions of boundary discovery, coverage, and total cleaning times side-by-side.

### Key Outcomes

#### Coverage of Inset Rectangle:

This is the most crucial metric to demonstrate the efficacy of the deskinator, as it measures the swept 'safe' area by the fan. This area is calculated by applying a margin equal to the distance between the sensor line and the start of the fan line, and then used to calculate the boustrophedon path. This ensures that the robot doesn't attempt to step outside of the table area it has identified. Therefore, this is effectively the only area we can safely sweep. Testing shows a range of values (% of area swept) between 96,95% and 100%, with a mean of 99.40% and standard deviation of

0.70%, indicating highly effective sweeping of the reachable area of the table, therefore meeting the main Key Objective of the project.

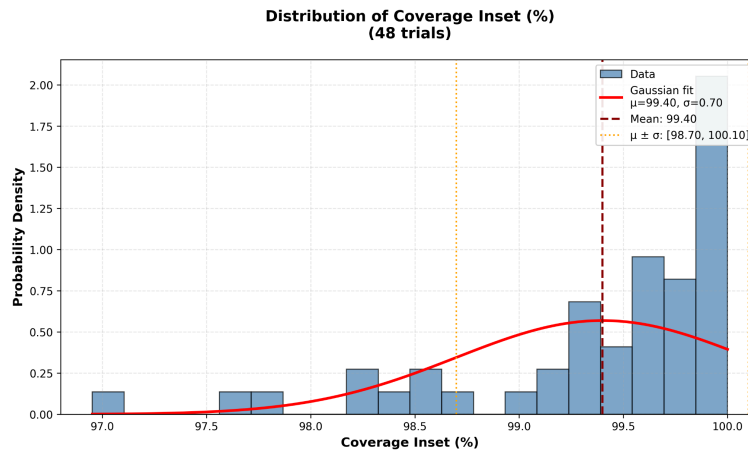


Figure 4: Graph of Distribution of Coverage of Safety Inset Rectangle with Gaussian fitting

### Time Metrics:

Another crucial aspect of the successful and efficient implementation of the deskinator is its speed. One of the Key Objectives was for the full sweeping of the table to be completed in under 120 seconds. The conducted testing used conservative linear and angular velocity limits (see config.py) but still achieved a mean completion time of 137 seconds, a standard deviation of 3.57 seconds, and a maximum value of 146,25 seconds. Increasing the linear and angular velocity limits by just 21.875% would reduce maximum times to 120 seconds, as well as significantly decreasing the mean time. Testing with higher limits has been conducted and showed improved speed performance, but lower limits were chosen for repeated performance due to concerns linked to overheating of the A4988 motor drivers. The addition of a heatsink would solve these issues, effectively decreasing mean operating time.

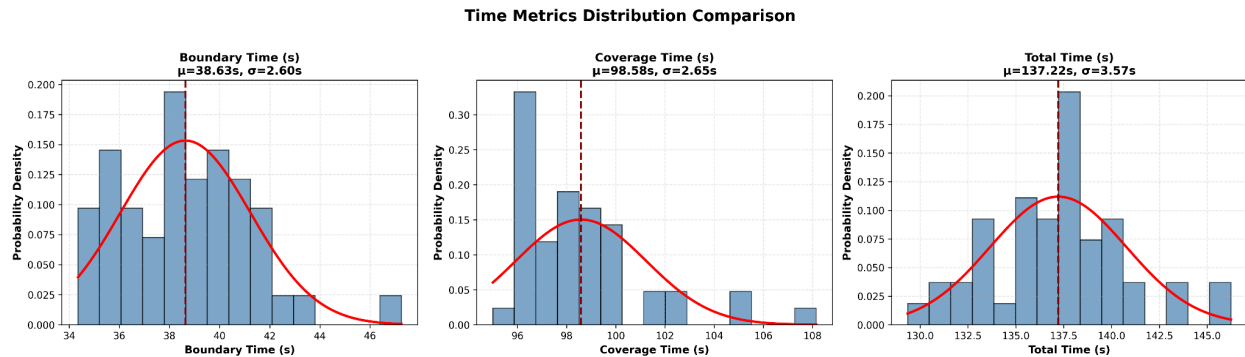


Figure 5: Graphs of Distribution of Time to complete boundary, discovery and total times, with Gaussian fitting

## Lessons

### I2C Software-Hardware Timing Issues

During the first prototype iteration, an I<sup>2</sup>C expander MUX was used to manage what were initially four APDS9960 sensors for edge detection. The MUX also carried the IMU and an additional APDS9960 for gesture sensing, giving a total of six I<sup>2</sup>C devices connected in parallel to a single hardware I<sup>2</sup>C port (GPIO 2–3 on the Raspberry Pi 4B). This arrangement produced unusable data because of timing conflicts on the bus.

The Raspberry Pi's default hardware-I<sup>2</sup>C clock rate is 100 kHz. Since the sensors were polled serially, the effective bus load rose significantly. This likely violated timing requirements, specifically when the effective clock period became smaller than the internal sensor timing constraints ( $t_{\text{clk}} < t_{\text{clk-q}} + t_{\text{logic}} + t_{\text{setup}}$ ). As a result, the sensor frames arriving at the Pi were corrupted.

To address this, we attempted lowering the hardware I<sup>2</sup>C clock in config.txt by adjusting the `i2c_arm_baudrate` of bus 1. Documentation and community reports reported that although the APDS9960 interface can technically support up to ~200 kHz, the internal proximity engines update at only ~100 Hz. Achieving stable communication required lowering the bus speed far below the Pi's default. Although this produced clean data, the resulting latency was too large, causing delayed edge detection and occasionally having the front ball caster falling off the table.

The successful solution was to transition to the Raspberry Pi's software-I<sup>2</sup>C capability, which allows additional virtual I<sup>2</sup>C buses to be instantiated on arbitrary GPIO pins. These software buses operate reliably around 10–50 kHz depending on the bit-bang delay. We reduced the edge-detection system to two front-facing sensors (switching to a wall-following algorithm) and moved the remaining sensors onto three separate software I<sup>2</sup>C buses. The gesture sensor remained on the hardware bus to preserve responsiveness and to have the two front sensors on similar hardware-software setups.

The following overlays were added to boot/firmware/config.txt:

```
dtoverlay=i2c-gpio,bus=5,i2c_gpio_sda=6,i2c_gpio_scl=13,i2c_gpio_delay_us=50
```

```
dtoverlay=i2c-gpio,bus=7,i2c_gpio_sda=19,i2c_gpio_scl=26,i2c_gpio_delay_us=50
```

```
dtoverlay=i2c-gpio,bus=3,i2c_gpio_sda=24,i2c_gpio_scl=25,i2c_gpio_delay_us=50
```

With `i2c_gpio_delay_us=50`, the software I<sup>2</sup>C bit-bang frequency is approximately:  $f \approx 1 / (2 \times \text{delay}) = 1 / (2 \times 50 \mu\text{s}) \approx 10 \text{ kHz}$ . This configuration completely resolved the data-integrity issues and produced clean, stable measurements across the APDS9960s and the MPU6050, enabling reliable edge detection, gesture sensing, and inertial tracking.

### Driver Current Limits and Microstepping Stability

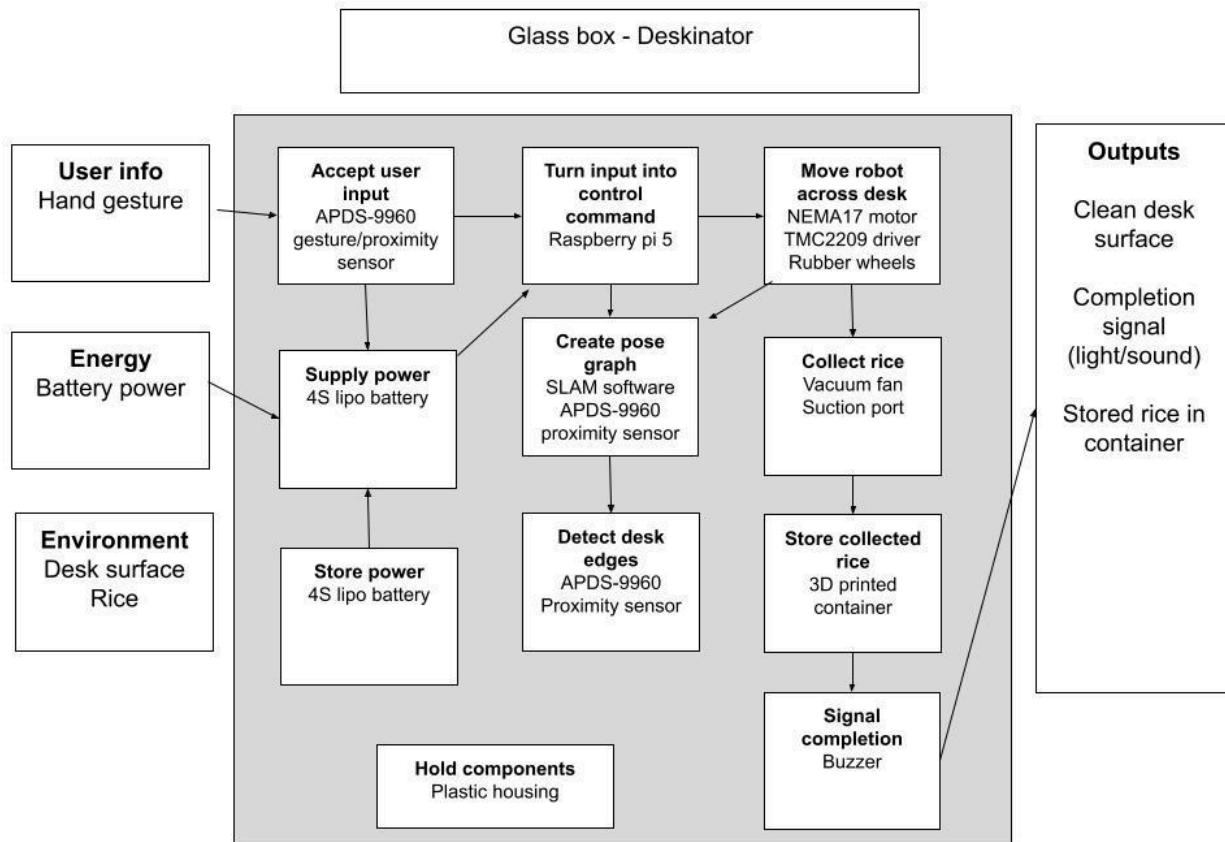
In the first prototype we reused an Adafruit DC & Stepper Motor HAT (TB6612FNG). The TB6612 datasheet caps each H-bridge at 1.2 A continuous, 3 A peak, with no chopper current control. Our NEMA17s are rated  $\sim 1.4\text{--}1.7$  A/phase with low coil voltage ( $\sim 3$  V), so driving them from a 12 V supply depended on the HAT's PWM and coil inductance to self-limit. That worked for a few seconds, but as soon as we asked for “microstepping” the bridge hit its over-current/thermal limits: coil current drooped, the current vector collapsed toward full-step, and the motors audibly jittered and lost position. Scope traces on the coil showed the PWM duty shrinking after a short warm-up, matching the TB6612 thermal foldback behavior noted in the datasheet. The lesson was that a voltage-mode H-bridge without a per-phase current limit will not hold a sine profile on a motor that wants more current than the bridge can supply.

We replaced the HAT with two A4988 drivers. The A4988 datasheet gives 35 V max VMOT and up to 2 A/phase with cooling, and—most importantly—a programmable current limit ( $I_{\text{trip}} \approx V_{\text{ref}} / (8 \cdot R_{\text{sense}})$ ). We set  $V_{\text{ref}}$  for  $\sim 1$  A/phase to stay under the NEMA17 rating while keeping the motors cool on a 12 V rail. The A4988's fixed 1/16-step microstep table keeps the phase currents sinusoidal regardless of supply voltage, and its built-in decay/chopper logic prevents the thermal collapse we saw on the TB6612. The board-level electrolytic caps also tame the VMOT spikes we observed when the HAT dumped current during stalls.

In software we leaned into the A4988's requirements: we honor the  $1\text{ }\mu\text{s}$  STEP pulse spec with a  $10\text{ }\mu\text{s}$  pulse, cap the step frequency to 10 kHz, and ramp velocity with acceleration limits so we don't out-pace the current regulator. Microstepping is baked into the odometry, so we keep the resolution gains without losing steps. This stack (current-limited A4988 + timing- and acceleration-aware drive code) eliminated the mid-run jitter and gave us smooth, repeatable microstepping at the speeds needed for desk-safe motion.

## Appendix

### A.1 Glass Box Diagram



## A.2 PCC Table

| objectives             | # | accuracy | # | percision | # | affordability | easy to clean | durability | mobility | portable | easy to use | reliable | navigation | completion recognition | touchless | storage capacity | autonomy | compactness | efficiency | total |
|------------------------|---|----------|---|-----------|---|---------------|---------------|------------|----------|----------|-------------|----------|------------|------------------------|-----------|------------------|----------|-------------|------------|-------|
| accuracy               |   |          |   | 1         |   | 1             | 1             | 1          | 1        | 1        | 1           | 1        | 0          | 1                      | 1         | 1                | 1        | 1           | 1          | 14    |
| percision              | 0 |          |   |           |   | 1             | 1             | 0          | 0        | 0        | 1           | 0        | 1          | 1                      | 0         | 1                | 0        | 0           | 0          | 6     |
| affordability          | 0 | 0        |   |           |   |               | 1             | 0          | 0        | 0        | 1           | 0        | 0          | 1                      | 0         | 0                | 0        | 1           | 0          | 4     |
| easy to clean          | 0 | 0        | 0 |           |   |               |               | 0          | 0        | 0        | 0           | 0        | 0          | 0                      | 1         | 0                | 1        | 0           | 0          | 3     |
| durability             | 0 | 1        | 1 |           |   |               | 1             |            | 0        | 1        | 1           | 0        | 0          | 1                      | 0         | 0                | 0        | 1           | 0          | 7     |
| mobility               | 0 | 1        | 1 | 1         |   |               | 1             | 1          |          | 1        | 1           | 0        | 0          | 1                      | 1         | 1                | 0        | 1           | 0          | 10    |
| portable               | 0 | 1        | 1 | 1         | 1 |               | 1             | 0          | 0        |          | 0           | 0        | 0          | 0                      | 0         | 0                | 0        | 1           | 0          | 4     |
| easy to use            | 0 | 0        | 0 | 0         | 1 | 0             | 0             | 0          | 1        |          |             | 0        | 1          | 1                      | 1         | 1                | 0        | 1           | 0          | 7     |
| reliable               | 1 | 1        | 1 | 1         | 1 | 1             | 1             | 1          | 1        | 1        | 1           |          | 1          | 1                      | 1         | 1                | 1        | 1           | 1          | 15    |
| navigation             | 0 | 0        | 0 | 1         | 1 | 1             | 1             | 1          | 1        | 0        | 0           |          |            | 1                      | 1         | 1                | 0        | 1           | 1          | 10    |
| completion recognition | 0 | 0        | 0 | 0         | 0 | 0             | 0             | 0          | 0        | 1        | 0           | 0        | 0          |                        | 0         | 0                | 0        | 0           | 0          | 1     |
| touchless              | 0 | 1        | 1 | 1         | 1 | 0             | 1             | 0          | 1        | 0        | 0           | 0        | 0          | 1                      |           | 0                | 0        | 1           | 0          | 7     |
| storage capacity       | 0 | 0        | 0 | 1         | 0 | 1             | 0             | 1          | 1        | 0        | 0           | 1        | 0          | 1                      | 1         |                  | 0        | 1           | 0          | 7     |
| autonomy               | 0 | 0        | 0 | 1         | 1 | 1             | 1             | 1          | 1        | 1        | 0           | 0        | 1          | 1                      | 1         | 1                |          | 1           | 0          | 10    |
| compactness            | 0 | 1        | 0 | 0         | 0 | 0             | 0             | 0          | 0        | 0        | 0           | 0        | 0          | 1                      | 0         | 0                | 0        |             | 0          | 2     |
| efficiency             | 0 | 1        | 0 | 0         | 1 | 1             | 1             | 1          | 1        | 1        | 1           | 0        | 0          | 1                      | 1         | 1                | 1        | 1           |            | 11    |

## A.3 Morph Chart

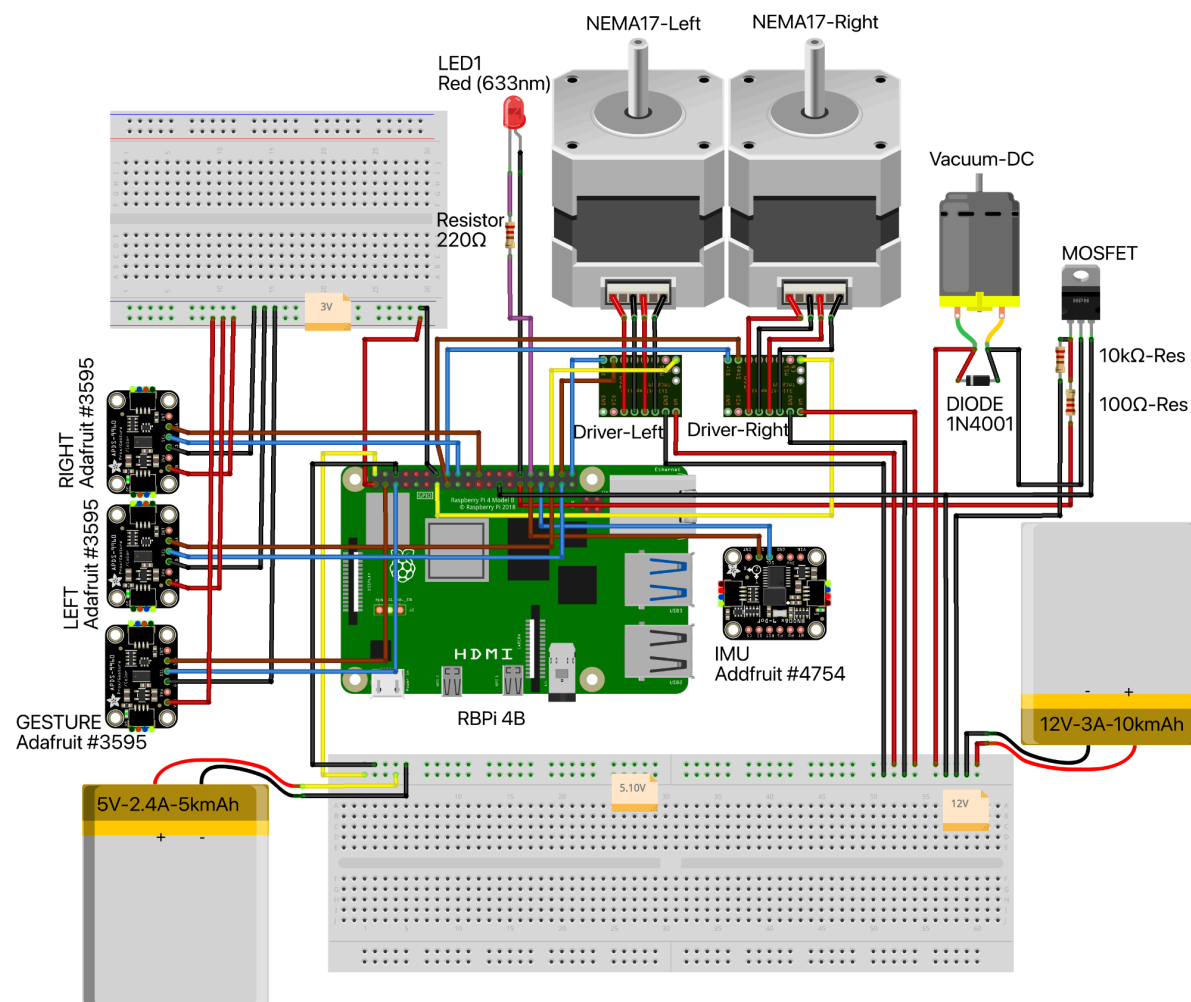
| Function              | Means 1                                          | Means 2                                       | Means 3                                                  |
|-----------------------|--------------------------------------------------|-----------------------------------------------|----------------------------------------------------------|
| <b>Edge Detection</b> | Proximity sensors facing down towards the table. | Contact sensors with probs towards the table. | Infrared distance sensors facing down towards the table. |
| <b>Vacuum Suction</b> | Small axial fan with funnel                      | Centrifugal impeller with dust box            | Piezo microblower                                        |
| <b>Motion</b>         | 2-wheel differential drive                       | 4-wheel skid steer                            | Continuous track (tank style)                            |
| <b>Start Signal</b>   | Motion detection                                 | Clap detection                                | Light signal                                             |
| <b>End Indicator</b>  | Buzzer                                           | LED light                                     | Screen display                                           |

## A.4 Bill of Materials

| Label           | Part Type     | Price (\$) |
|-----------------|---------------|------------|
| <b>100Ω-Res</b> | 10kΩ Resistor | 0          |
| <b>10kΩ-Res</b> | 10kΩ Resistor | 0          |

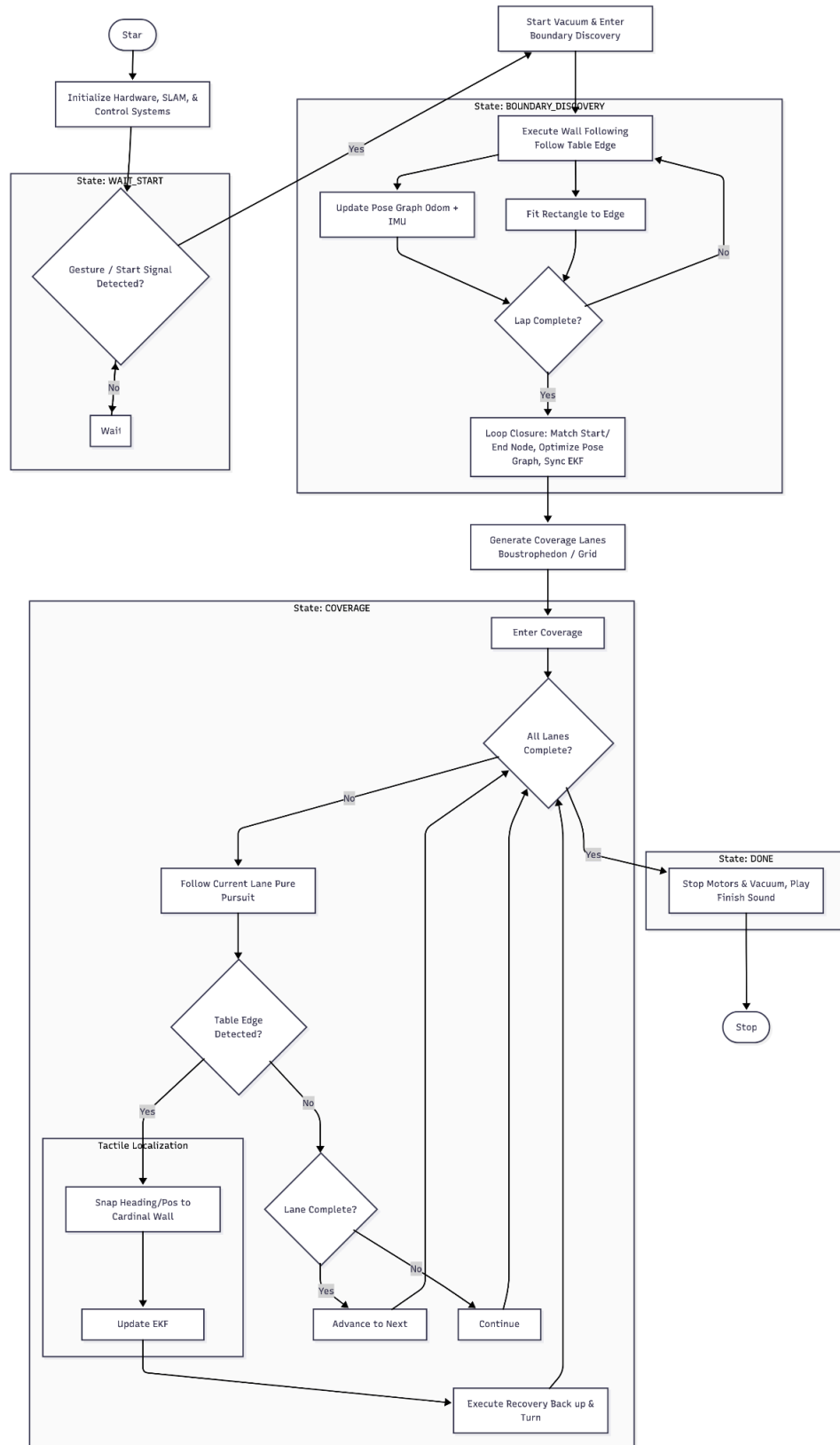
|                      |                                    |                   |
|----------------------|------------------------------------|-------------------|
| <b>12V-3A-10kmAh</b> | 12V LIPO                           | 18,48             |
| <b>5V-2.4A-5kmAh</b> | 5V LIPO                            | 18,48             |
| <b>DIODE</b>         | Rectifier Diode                    | 0                 |
| <b>Driver-Left</b>   | Adafruit #6109 A4988               | 6,95              |
| <b>Driver-Right</b>  | Adafruit #6109 A4988               | 6,95              |
| <b>GESTURE</b>       | Adafruit APDS9960 Proximity Sensor | 7,5               |
| <b>IMU</b>           | MPU6050                            | 6,99              |
| <b>LED1</b>          | Red (633nm) LED                    | 0                 |
| <b>LEFT</b>          | Adafruit APDS9960 Proximity Sensor | 7,5               |
| <b>MOSFET</b>        | MOSFET-NCHANNEL                    | 0                 |
| <b>NEMA17-Left</b>   | NEMA 17 P/N: 17HS15-1504S-X1       | 9,13              |
| <b>NEMA17-Right</b>  | NEMA 17 P/N: 17HS15-1504S-X1       | 9,13              |
| <b>RBPi 4B</b>       | Raspberry Pi 4B                    | 35                |
| <b>Resistor</b>      | 220Ω Resistor                      | 0                 |
| <b>RIGHT</b>         | Adafruit APDS9960 Proximity Sensor | 7,5               |
| <b>Vacuum-DC</b>     | DC Vacuum Fan 12V                  | 25                |
| <b>TOTALS</b>        | <b>18</b>                          | <b>158,61</b>     |
| <b>Label</b>         | <b>Part Type</b>                   | <b>Price (\$)</b> |

## A.5 Electronics Sketch



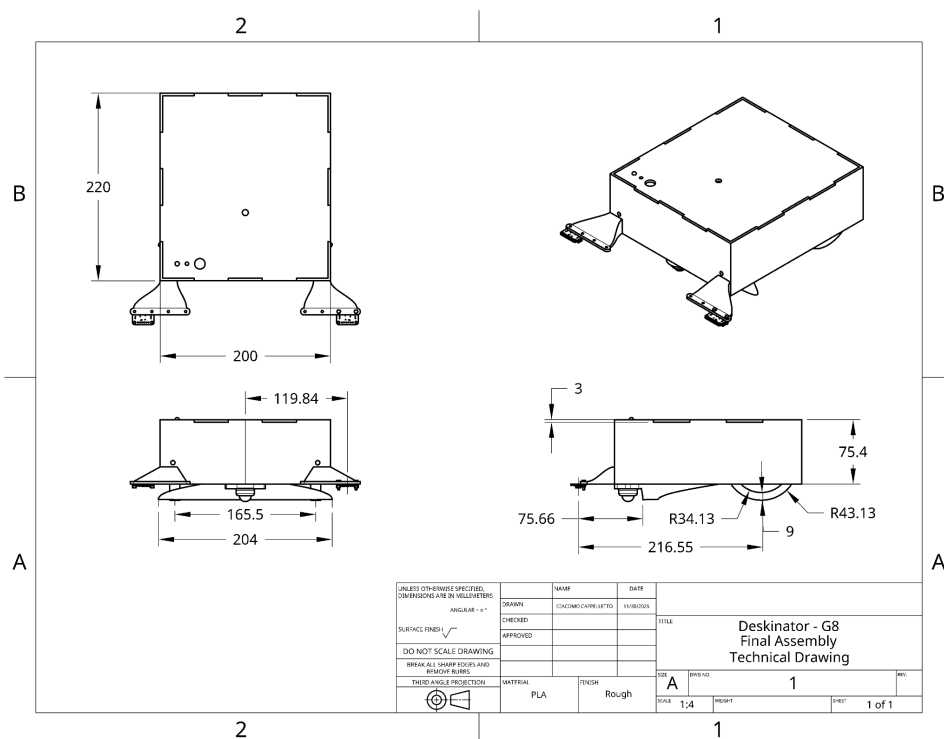
fritzing

## A.6 Code Flow Chart

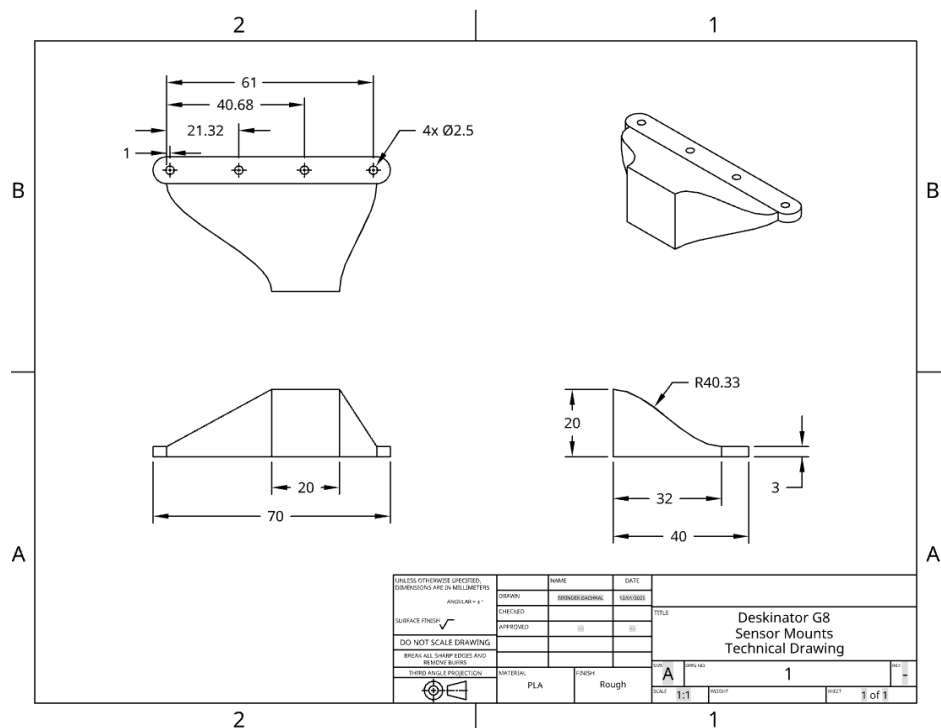


## A.7 CAD Drawings

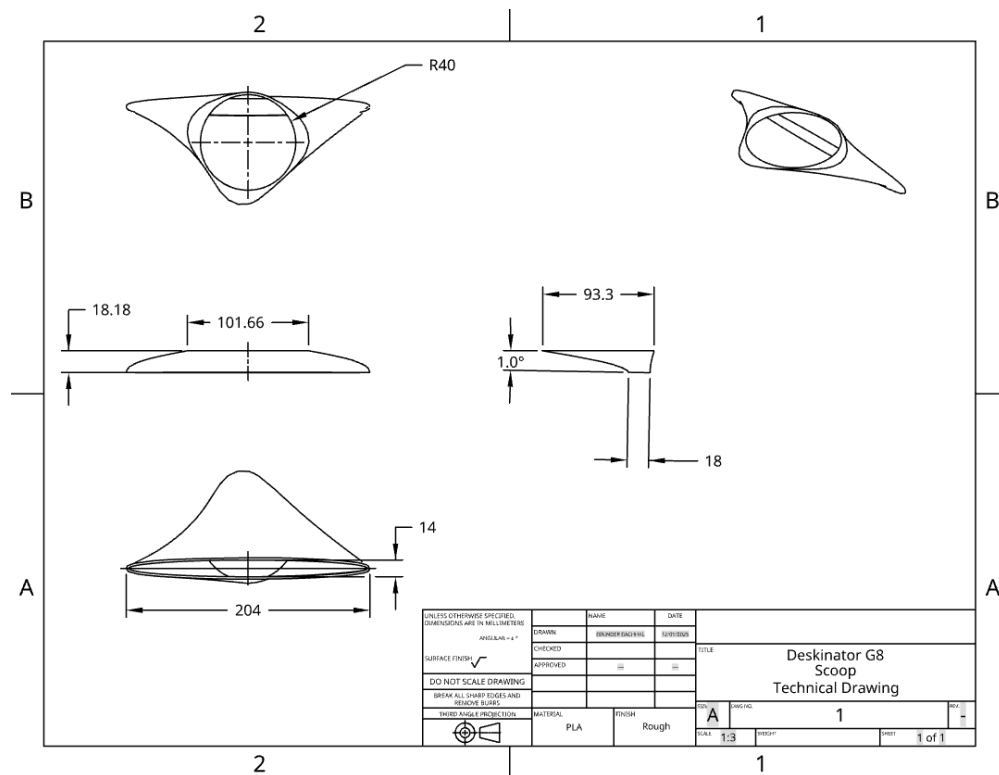
### Main Assembly



### Sensor Mounts



## Scoop



## A.8 Power budget calculations

### 12 V System (Motors + Fan)

- Battery: 12 V, 10 000 mAh → 120 Wh
- Loads:
  - Fan:  $12\text{ V} \times 1\text{ A} = 12\text{ W}$
  - $2 \times \text{NEMA17: } 10\text{ V} \times 1\text{ A each} = 20\text{ W}$
  - Total:  $\sim 32\text{ W}$
- Ideal runtime:  $120\text{ Wh} / 32\text{ W} \approx 3.75\text{ h}$
- **Realistic (derating + driver losses):  $\sim 3.0\text{ h}$**

### 5 V System (Raspberry Pi + Sensors)

- Battery: 5 V, 5000 mAh → 25 Wh
- Loads:
  - Raspberry Pi 4B:  $\sim 5\text{--}8\text{ W}$

- $\text{APDS9960} \times 3 + \text{MPU6050}: \sim 0.1 \text{ W}$
- Runtime:  $25 \text{ Wh} / (5\text{--}8 \text{ W}) \approx 3.1\text{--}5.0 \text{ h}$
- **Realistic:  $\sim 3.5\text{--}4.0 \text{ h}$**

### Overall Robot Runtime

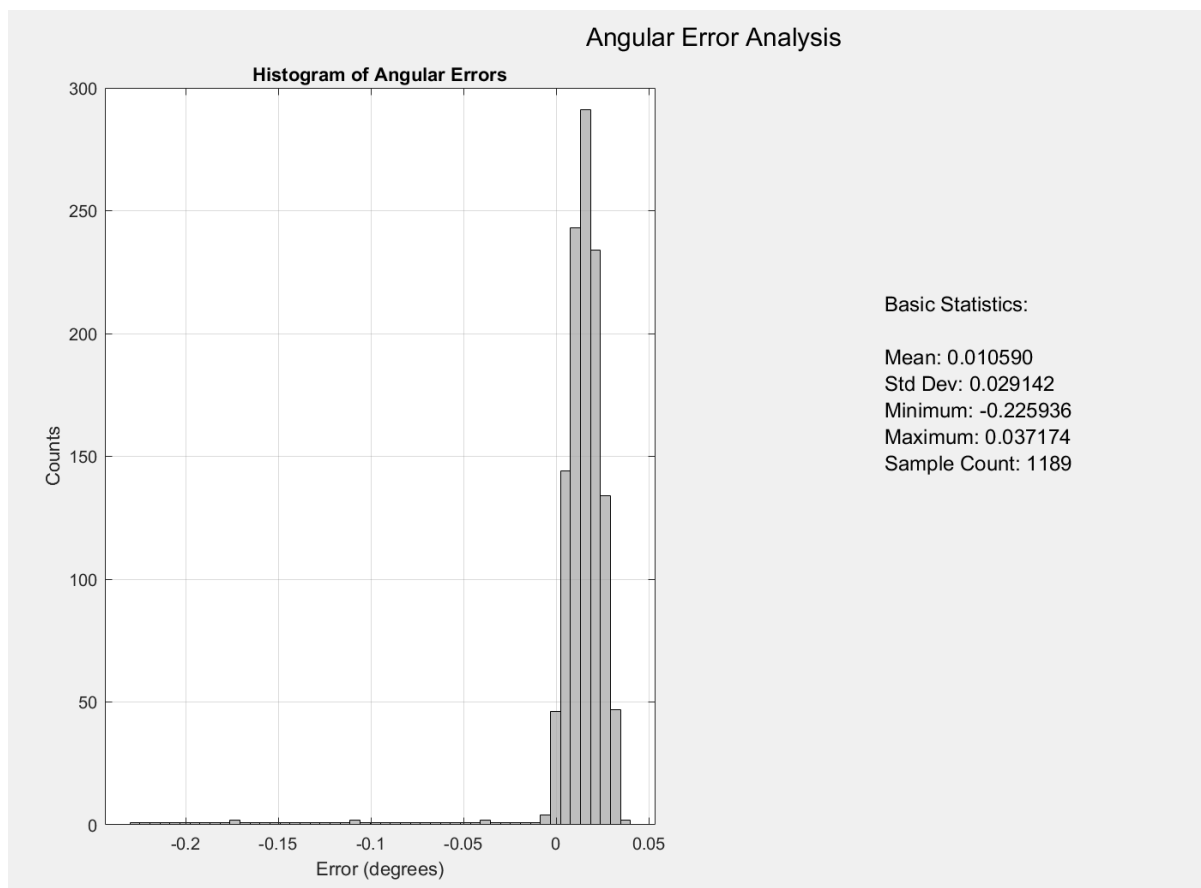
- Limited by the 12 V system.
- **Estimated continuous operating time:  $\sim 3$  hours.**

### A.9 Code

[Github](#)

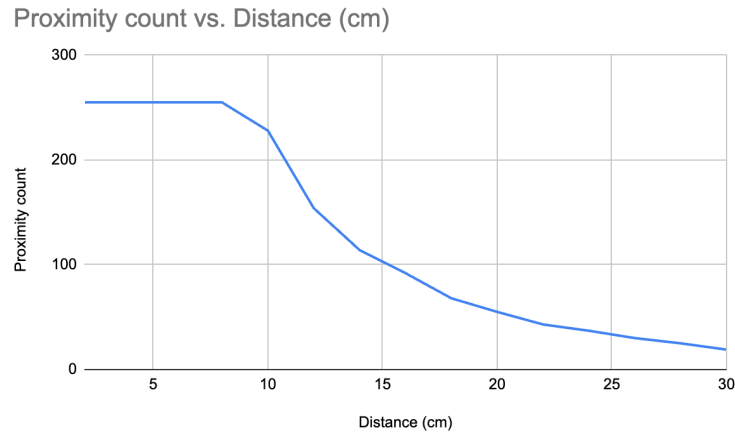
### A.10... Plots and tables related to your testing

#### Actuator Testing



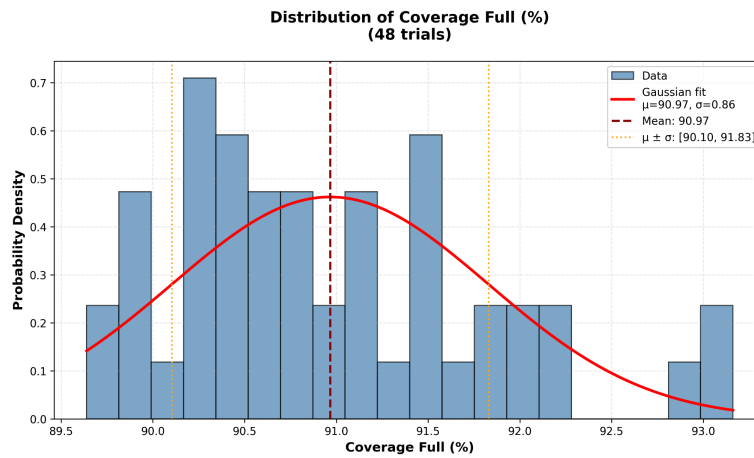
*Figure: Precision of the Nema-17 stepper motor; showing error distribution and basic statistics regarding angular error found from Nema-17 stepper motor*

### Sensor Testing



*Figure: APDS 9960 Proximity Sensor's ability to detect an object at various distances*

### Integrated Testing



*Figure: Graph of Distribution of Coverage of Full table area with no safety inset edge*

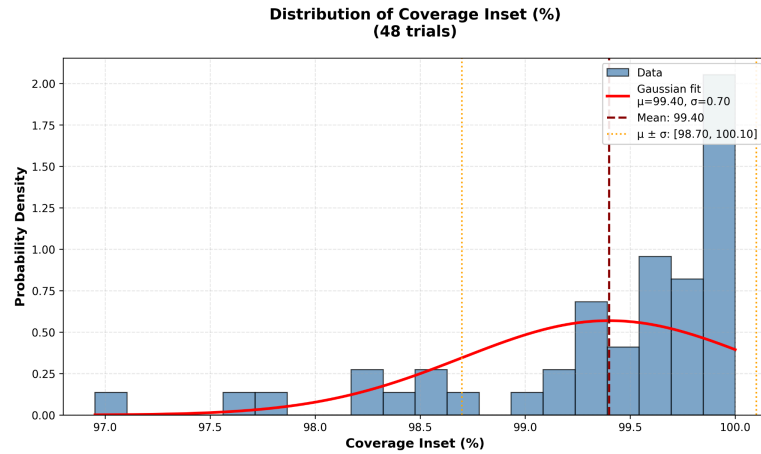


Figure: Graph of Distribution of Coverage of Safety Inset Rectangle with Gaussian fitting

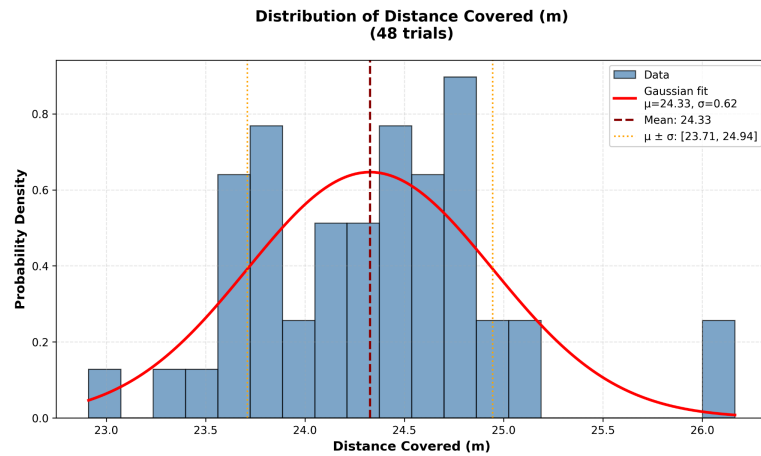


Figure: Graph of Distribution of Covered distance by robot during process

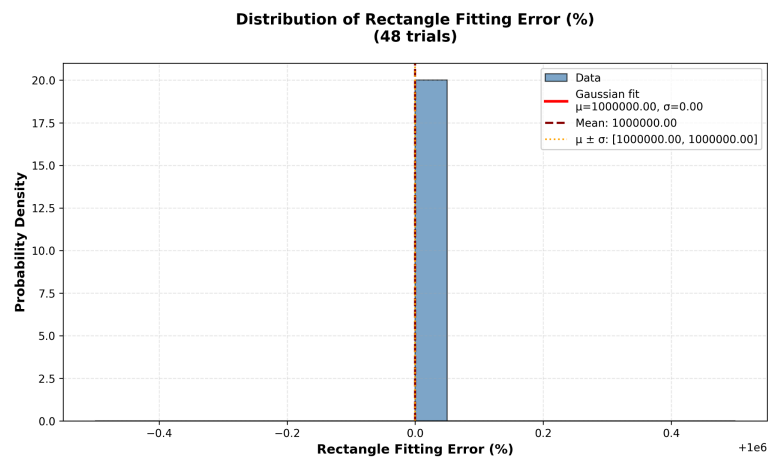
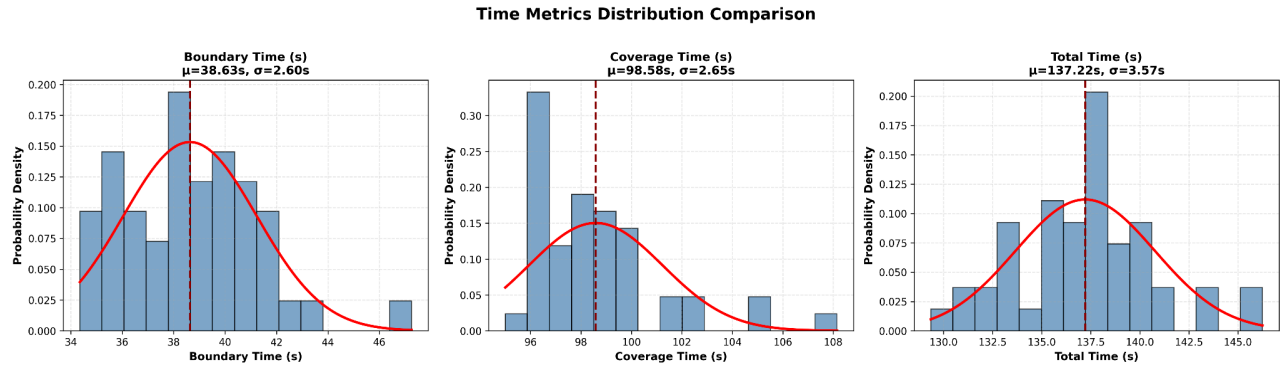


Figure: Graph of Distribution of Rectangle fitting error from edge points ( $10^6\text{ppm} = 0\%$ , perfect fit)



*Figure: Graphs of Distribution of Time to complete boundary, discovery and total times, with Gaussian fitting*